NFS Lab 1: Wireshark Introduction and Network Analysis Lab

I Lab Setup

This lab introduces practical network observation using command line tools and Wireshark. The objective is to observe how common network protocols operate in practice, by capturing and analyzing real traffic generated by your machine.

We will perform most of the manipulations by allowing you to be root when needed, for example, to manipulate network setup, monitor trafic, install, modify and run software that requires specific permissions.

I.1 Machine Access

You can do this lab either on your personal computer or using the machine in the lab rooms.

If you choose the second option, you will be able to connect using the password:

```
!?2+en+dur?!
```

For easy of setup, you will use VirtualBox to run a linux VM. Please launch Virtualbox and import the linux image from **/VM-Source/** . If you don't see the file, just drag and drop the VM in the software.

II Network Observation

In this part, we will learn basic commands to check or modify the network setup and interrogate network infrastructure. Here is a cheat sheet for you to help you with the commands.

Usage	**Command**
Visualize or modify the ARP table	`ip neigh` (or `arp`)
Visualize or modify your interfaces	`ip addr`, `ip link` (or `ifconfig`)
Visualize or modify your routes	`ip route` (or `route`, or `netstat -r`)
Send Domain Name Server (DNS) requests	`dig`
Send an ICMP echo request (ping)	`ping`
Visualize the path to a remote host	`traceroute`
Display information about open connections	`ss` (or `netstat`)
Dump traffic	`tcpdump`, `tshark`, or `wireshark` (GUI)
Read information about commands	`$command -h` or `man $command`

II.1 Understanding the local network

1. Find the IP address of your main network interface.

2. Display your routing table. You will likely have two or three routes.

3. Display your neighbors.

4. If no neighbor is visible, send a ping to a host such as `google.fr` and check the neighbor table again.

5. Remove the neighbor entries and send another ping. Check the neighbor table once more.

At this point, you should be able to explain:

- Which interface is used to reach the Internet?
- What are the different routes in your routing table, and why are they needed?
- What information is stored in a neighbor entry?
- Why does a neighbor entry reappear after you remove it?

II.2 Outside your network

6. Find the IP address used by google.com, using the proposed command.

7. Investigate the path your computer uses to reach:
- Your local machine: localhost (or IP 127.0.0.1)
- univ-rennes.fr
- youtube.com
- Any site you like.

8. Run a traceroute from a remote machine to compare the results.

```
There are many websites you can use to run traceroutes or other network debugging tools to
investigate networking from other networks than your own. You can use the following
website :

- https://lg.tetaneutral.net, then select traceroute instead of show protocols on the top
menu.

Send a traceroute to the previously tested addresses (excluding localhost). These servers,
named “looking glass”, allow you to run network debugging commands from other machines.
```

9. Compare the paths observed from your machine and from the remote machine.

You should be able to explain:

- What the hops displayed by `traceroute` represent.
- Why the path can be different depending on where the traceroute is launched.
- Why some parts of the two paths may nevertheless be identical or similar.

Checkpoint 1: Call your lab supervisor to explain your findings.

III Network Monitoring with Wireshark

Network traffic is composed of packets exchanged continuously between machines. Wireshark allows us to capture and inspect these packets in real time.

In this section, you will observe a simple web request and identify the main protocols involved.

- Launch Wireshark.
- Identify the active network interface.
- Start capturing packets on this interface.

III.1 Address Resolution Protocol (ARP)

10. Start a new Wireshark capture and filter it to display only ARP messages.

11. First, check your current ARP/neighbor table, then flush the neighbor table.

```
$~ ip neigh
$~ ip neigh flush all
```

12. Now generate some network traffic, for example by visiting a website.

13. Observe the packets captured by Wireshark and identify the ARP request and ARP reply. Check your neighbor table again.

Could you explain:

- What IP address is being resolved?
- What MAC address is being discovered?
- Why does the ARP request need to be sent again after flushing the table?
- Why is the target MAC address not known in the request?

III.2 Dynamic Host Configuration Protocol (DHCP)

Filter your traffic to display only DHCP messages.

In a terminal, disconnect your network interface and reconnect it using:

```
$~ nmcli device disconnect <interface> # of course replace <interface> with the right one
$~ nmcli device connect <interface>
```

Explore the packets and determine:

- Which machine sends each message?
- Which addresses are used as source and destination?
- What information does the client request?
- What information does the DHCP server provide?

Find:

- the IP address assigned to your machine;
- the subnet mask or network prefix;
- the default gateway;
- the DHCP server address;
- the lease duration, if available.

Do not only look at the packet summary. Open the DHCP fields and inspect the options contained in the messages.

Checkpoint 2: Call your lab supervisor and explain your findings.

III.3 Bonus: Transport Control Protocol (TCP)

If you want more let start with the future lecture. You will investigate how TCP establishes a connection and how Wireshark allows you to isolate a single TCP conversation from all the other traffic being captured.

Open a simple HTTP-only website (without HTTPS) such as http://httpforever.com/ and start a Wireshark capture.

14. Apply the display filter: `tcp.flags.syn == 1 || tcp.flags.ack == 1`.

Find the beginning of a TCP connection. How many packets are exchanged before application data starts appearing? What do you observe in the TCP flags of these packets?

15. Right-click on one of the packets and select `Follow → TCP Stream`. Alternatively, identify the stream number and use `tcp.stream == N`.

Explore the stream and identify the HTTP request and response. How does Wireshark know which packets belong to this conversation?

16. Look at the first packets of the TCP stream and the packets containing application data. What is the purpose of the first three packets?